

GraspGraphNet: Graph-Structured Multi-Embodiment Dexterous Grasp Generation

Abstract—Dexterous grasp generation across robot hands is challenging because hands differ in kinematic topology, actuation dimensions, and native command spaces. We introduce GraspGraphNet, a topology-aware grasp generation framework that represents each hand as a URDF-derived kinematic graph and directly generates executable palm poses and joint configurations. GraspGraphNet combines hierarchical object surface encoding, differentiable forward kinematics, and dynamic world-edge message passing to model evolving robot-object interactions. It applies conditional flow matching directly in executable palm-pose and joint-state space, avoiding post-processing optimization, inverse kinematics, and retargeting. Using a shared model trained on Barrett Hand, Allegro Hand, and Shadow Hand, GraspGraphNet achieves an average success rate of 83.48% with 0.040 s inference time per grasp on a 40-object benchmark. Without retraining, the same model achieves 72.70% success on controlled finger-removal variants, demonstrating robustness to hand-topology variations. These results suggest that graph-structured hand representations can effectively support dexterous grasp generation across robot hands with different kinematic structures.

I. INTRODUCTION

DEXTEROUS grasping is a fundamental capability for robotic manipulation, requiring a multi-fingered robot hand to coordinate high-dimensional joint motion while establishing physically meaningful contact with object surfaces. Recent learning-based methods have made substantial progress in generating stable and diverse grasps for dexterous hands and complex object geometries [1]–[5]. Despite this progress, scaling dexterous grasp generation across diverse robot hands remains challenging. Dexterous Grasp generation must consider object geometry as well as hand-specific kinematics, link geometry, joint limits, and actuation spaces.

A key challenge is that different dexterous robot hands do not share a common configuration space. Robot hands may have different numbers of fingers, links, joints, and kinematic trees. As a result, both the robot input and the joint command output vary across embodiments. This makes it difficult to design a single grasping model that supports multiple dexterous hands. We define cross-embodiment grasp generation as learning a unified model across multiple robot-hand embodiments with distinct kinematic structures and actuation spaces.

To address this challenge, prior learning-based grasp generation methods have explored transferable representations that reduce dependence on a specific robot configuration space. Some methods introduce hand-agnostic grasp cues, such as object contacts [6], [7], contact maps [3], [8], or contact-centric representations [5], [9], [10]. These representations are transferred to different hands through downstream optimization, retargeting, or hand-specific decision modules. Another

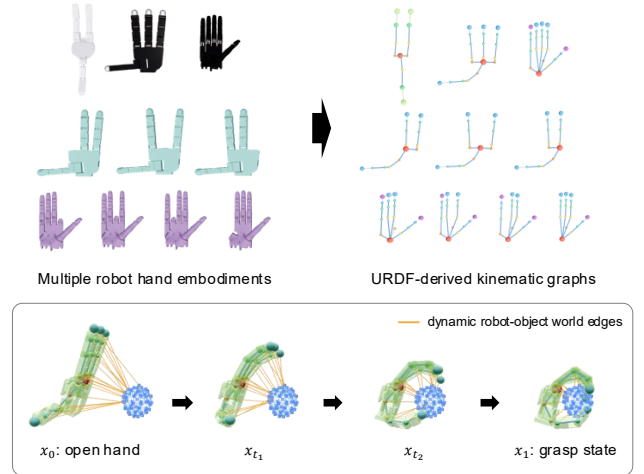


Fig. 1. GraspGraphNet overview. Robot hands are represented as URDF-derived kinematic graphs, while dynamic robot-object edges evolve with the grasp state from x_0 to x_1 .

class of methods explicitly models robot-object interaction: DRO-Grasp [11] represents dense robot-object distances, while TRO-Grasp [12] generates robot-object spatial transformations with graph diffusion. While such contact and interaction-level representations improve transfer across hands, they cannot be directly executed on a robot hand and must be converted into feasible palm poses and joint commands. This conversion commonly requires post-processing optimization, inverse kinematics, or retargeting, which can increase inference cost and introduce sensitivity to hand kinematics, joint limits, and initialization.

These limitations motivate **GraspGraphNet**, a topology-aware dexterous grasp generator based on robot-object graph representations. GraspGraphNet directly generates the palm pose and actuated joint configuration in the configuration space of the input hand.

A robot hand described by a URDF naturally forms an articulated kinematic tree [13], in which physical links are connected by joints. We represent this URDF structure as a link-joint graph and encode the target object as a hierarchical surface graph for multi-scale geometric reasoning. The robot graph preserves the articulated structure of the hand, allowing information to propagate along kinematic connections rather than representing the robot state as an unordered or fixed-length vector. Graph-based computation enables a single parameter-sharing model to process trained hand embodiments with different graph sizes, connectivities, and joint dimensions [14]–[16]. As a result, the model can be directly applied to controlled topology variants without retraining. Rather

than predicting a hand-agnostic representation that requires a separate reconstruction step, GraspGraphNet directly outputs the palm pose and joint state of the input hand. This keeps grasp generation aligned with each hand’s native kinematic structure and reduces inference latency.

Since robot-object proximity changes continuously during grasp formation, the relevant interaction neighborhoods should evolve with the current hand state. GraspGraphNet therefore constructs dynamic world-frame edges between articulated hand links and nearby object surface points, allowing robot-object interaction features to be updated throughout generation.

We formulate grasp synthesis as conditional flow matching in executable palm-pose and joint-state space. Starting from an open-hand initialization, the model learns a velocity field that transports the hand toward a final grasp configuration. Compared with iterative denoising formulations, flow matching enables efficient trajectory integration with a small number of network evaluations, making it well suited for direct executable-state generation [17].

We compare GraspGraphNet with prior cross-embodiment methods on Barrett, Allegro, and Shadow hands, which span three to five fingers and different kinematic structures. While prior evaluations commonly report results on a small set of validation objects, we additionally include 30 objects from Google Scanned Objects [18], resulting in a 40-object evaluation covering a broader range of object geometries. We further compare GraspGraphNet with prior cross-embodiment methods on controlled finger-removal variants that alter the hand graph, link set, and actuated joint dimension. This evaluation measures robustness to structural variations without retraining. Experimental results demonstrate that GraspGraphNet achieves strong grasp success rates, efficient inference, and favorable robustness under both expanded object evaluation and finger-removal settings.

The main contributions of this work are summarized as follows:

- We propose **GraspGraphNet**, a topology-aware dexterous grasp-generation framework that operates on URDF-derived hand graphs. It supports a unified model across multiple hands and direct application to controlled topology variants without retraining.
- We introduce a **dynamic world-edge message passing** mechanism that continuously updates robot-object interaction neighborhoods according to the current articulated hand configuration, enabling the model to capture evolving local contact geometry throughout grasp generation.
- We formulate dexterous grasp synthesis as **conditional flow matching in executable palm-pose and joint-state space**, allowing direct generation of robot-specific grasp commands without embodiment-specific post-processing.

II. RELATED WORK

A. Dexterous Grasp Generation

Dexterous grasp generation aims to synthesize stable grasp poses and joint configurations for multi-fingered robot hands.

Early optimization-based methods generate grasps by maximizing force-closure [19] or contact-quality objectives [20], but often require expensive test-time optimization and accurate object geometry. Recent learning-based approaches improve efficiency by learning grasp distributions from large-scale datasets [2]–[4].

Existing learning-based methods can be broadly categorized into robot-centric and object-centric approaches. Robot-centric methods directly predict executable grasp states such as wrist poses or joint values, enabling efficient execution but typically restricting the model to a specific hand embodiment and output space [21]–[24]. In contrast, object-centric approaches represent grasps through contact points, contact maps, or affordance-related cues, allowing better transfer across objects and robotic hands [5], [10], [25], [26]. However, these representations are not directly executable and generally require inverse kinematics, contact fitting, or optimization to recover feasible grasp configurations [3], [10], [25], [26]. This limitation motivates grasp representations that retain cross-embodiment transferability while enabling more direct grasp execution.

B. Cross-Embodiment Dexterous Grasping

Cross-embodiment dexterous grasping seeks a shared representation that can be applied across robot hands with different kinematic structures, morphologies, and output spaces. A major challenge is to make this representation transferable across embodiments while preserving robot-specific grasp feasibility.

Early approaches primarily relied on contact or geometry-based grasp representations. UniGrasp [25] and GeoMatch [26] predict contact points or keypoints conditioned on object and hand geometries, while GenDexGrasp [3] and CEDex [9] introduce a hand-agnostic contact-map representation for grasp transfer. RobotFingerPrint [10] instead proposes a unified coordinate space that establishes dense correspondences between gripper surfaces and object surfaces. These methods improve transferability by avoiding direct prediction in embodiment-specific joint spaces. Nevertheless, the predicted contacts, keypoints, or coordinate correspondences must still be converted into executable grasp states through inverse kinematics, optimization, or other hand-specific procedures.

Recent work has shifted toward interaction-centric representations that explicitly model robot-object relationships. DRO-Grasp [11] represents grasps through robot-object distance relationships, while TRO-Grasp [12] further introduces graph-based transformation representations between object patches and hand links. These methods demonstrate that robot-object interactions provide effective transferable representations across embodiments, but they still require a separate reconstruction or optimization stage to obtain executable robot commands, such as palm poses and joint configurations. Our work follows this interaction-centric direction but directly generates palm poses and joint configurations on a URDF-derived hand graph. Furthermore, robot-object interaction edges are dynamically updated during generation, enabling the interaction structure to evolve with the articulated hand state.

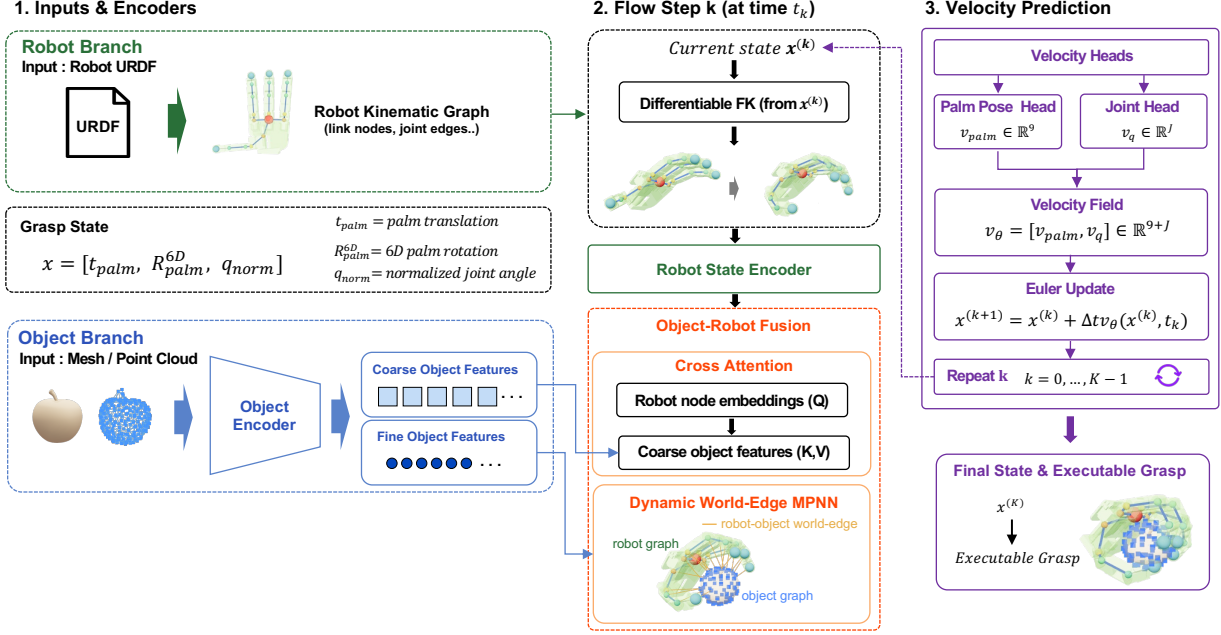


Fig. 2. Overview of GraspGraphNet. Given an object point cloud and a robot kinematic graph constructed from its URDF, GraspGraphNet generates an executable grasp by integrating a learned velocity field over the palm pose and joint state. At each flow step, differentiable forward kinematics computes the current world-space hand-link poses, and a kinematic-tree MPNN encodes the articulated robot state. The resulting robot node embeddings are fused with coarse and fine object features through cross-attention and dynamic world-edge message passing, respectively. Palm and joint velocity heads then predict the velocity field used to update the grasp state.

III. PROPOSED METHOD

A. Overview and Grasp-State Formulation

We propose GraspGraphNet, a topology-aware dexterous grasp generator that operates directly on object and robot graph representations. Given a target object point cloud and a robot URDF, the model generates an executable grasp consisting of the palm pose and actuated joint configuration of the input hand. Unlike methods that first predict an interaction representation and then convert it into robot commands, GraspGraphNet learns a conditional velocity field directly over the executable grasp-state space.

For a robot hand with J actuated joints, we represent a grasp state as

$$x = [t_{palm}, R_{palm}^{6D}, q_{norm}] \in \mathbb{R}^{9+J}, \quad (1)$$

where $t_{palm} \in \mathbb{R}^3$ is the palm translation in the object-centered frame, $R_{palm}^{6D} \in \mathbb{R}^6$ is the continuous 6D palm-orientation representation, and $q_{norm} \in \mathbb{R}^J$ contains the normalized actuated joint angles. To provide consistent orientation semantics across embodiments, the palm orientation is represented in a robot independent canonical palm frame. Each joint is normalized independently using the limits specified in the robot URDF. Because J is determined by the input robot graph, the grasp-state dimension can vary across hand embodiments.

As illustrated in Fig. 2, the object and robot representations condition a state-dependent vector field. At each flow step, the current state is articulated through differentiable forward kinematics, encoded over the robot kinematic tree, and fused with the object representation. Palm and joint velocity heads then predict an update to the current grasp state.

B. Object and Robot Graph Representations

GraspGraphNet uses complementary graph representations for the target object and the robot hand. The object representation captures geometry at multiple spatial resolutions, whereas the robot representation preserves the URDF-defined link-joint topology and link geometry of the input hand. Object points and state-conditioned robot poses are expressed in an object-centered world frame, while link geometry is initially represented in each link’s local frame.

1) *Object representation*: Given an object mesh or point cloud, we sample 1,024 surface points, each represented by its position, surface normal, and local curvature. We construct a four-level hierarchy with 1,024, 256, 64, and 32 points, where farthest-point sampling produces coarser levels and local features are aggregated from the preceding level [27]. A k -nearest-neighbor graph is built within each level for hierarchical point-graph message passing. We use the 64-point embeddings as fine object features O_f for contact-relevant geometry and the 32-point embeddings as coarse object features O_c for global shape context. Mean pooling the coarse features yields a global object descriptor for the grasp-velocity heads.

2) *Robot representation*: As illustrated in Fig. 3, we construct a robot kinematic graph $\mathcal{G}_R = (\mathcal{V}_R, \mathcal{E}_R)$ directly from the input URDF. Each node $i \in \mathcal{V}_R$ corresponds to a physical robot link, and each directed edge $(i, j) \in \mathcal{E}_R$ corresponds to a joint connecting a parent link i to a child link j . Virtual and robot-specific unused links are excluded. This representation naturally changes in graph size and connectivity across robot embodiments. Each kinematic edge stores its joint type, joint axis, joint limits, and fixed parent-to-child transformation. To

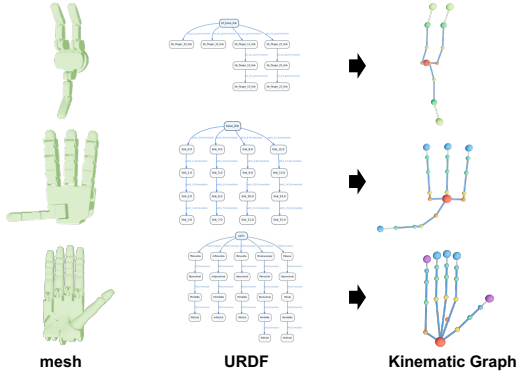


Fig. 3. URDF-derived robot hand representations. Physical links form graph nodes, and joints form directed parent-to-child edges. Each link is augmented with a geometry embedding extracted from points sampled on its visual mesh. The resulting graph preserves the embodiment-specific numbers of links, joints, fingers, and graph connectivity.

represent link geometry, we sample points from each visual mesh and encode them using a shared PointNet to obtain a per-link geometry embedding. These geometry embeddings are included in the state-conditioned robot node features.

C. State-Conditioned Robot Encoding

At flow time t , the current state x_t specifies a palm pose and normalized joint configuration. We denormalize the joint values using the URDF limits and apply differentiable forward kinematics over $\mathcal{G}_R$ to compute the world-space transformation T_i^t of each robot link. The link position p_i^t and orientation R_i^t are extracted from T_i^t . For each robot link, we construct a state-conditioned node feature by concatenating its link-geometry embedding, current FK-derived position and 6D orientation, kinematic-tree depth embedding, fingertip indicator, and the normalized state of its associated joint. For links not associated with an actuated joint, the joint-state input is set to zero. The concatenated feature is then projected to the robot-node embedding space using an MLP.

The projected node features are processed by a three-layer edge-conditioned MPNN over the URDF-derived kinematic edges. Messages are conditioned on the joint attributes stored on each kinematic edge. Consequently, the encoder captures the current articulated hand state while preserving the topology and link geometry of the input embodiment.

D. Object-Robot Fusion

1) *Coarse Cross-Attention*: The robot node embeddings serve as queries, while the coarse object features serve as keys and values. This operation conditions the articulated robot state on global object-shape context.

2) *Dynamic World-Edge Message Passing*: For each robot link, we then identify the K_W nearest fine object points and construct dynamic edges from these points to the robot node:

$$\mathcal{E}_W^t = \text{kNN}(\{p_j^{\text{obj}}\}, \{p_i^t\}). \quad (2)$$

Here, p_j^{obj} denotes a fine-level object point and p_i^t denotes the current world-space position of robot link i . This produces

$K_W|\mathcal{V}_R|$ directed edges and scales linearly with the number of robot links. Each message combines the fine object feature, current robot embedding, relative displacement, and distance. The messages from neighboring object points are aggregated at each robot node through residual update layers. Rebuilding $\mathcal{E}_W^t$ at every flow step allows the local interaction graph to evolve with the generated grasp trajectory.

E. Velocity Field and Flow-Matching Objective

1) *Velocity prediction*: The object-aware robot embeddings are mapped to palm and joint velocities. The palm pose and joint heads shown in Fig. 2 predict the palm-state velocity v_{palm} and per-joint velocity v_q , respectively. We designate the link node with the highest out-degree in the kinematic graph as the palm-hub node, which generally corresponds to the physical palm from which the finger chains branch. The palm velocity head reads the embedding of the designated palm-hub node, whereas the joint velocity head is shared across all actuated joints and is applied to the child-link embedding associated with each joint. Both heads are additionally conditioned on the global object descriptor and a sinusoidal time embedding. The scalar predictions are assembled according to the actuated-joint indices of the input graph to form $v_q \in \mathbb{R}^J$. The final grasp-state velocity is

$$v_\theta(x_t, t) = [v_{\text{palm}}; v_q] \in \mathbb{R}^{9+J}. \quad (3)$$

The shared per-joint head allows the same network to operate on hands with different actuation dimensions. We use x_t to denote a state at continuous flow time t during training, and $x^{(k)}$ to denote its numerical approximation at the k -th Euler step during inference, where $t_k = k/K$.

2) *Initial and target states*: For each training example, the target state x_1 is obtained from a ground-truth grasp annotation. The initial state x_0 is an open-hand pose sampled around the object, with all actuated joints initialized to the open-hand configuration. During training, we sample the initial palm approach direction within a cone centered at this target direction and sample the in-plane palm rotation uniformly over $[0, 2\pi)$.

When multiple ground-truth grasps are available for the same robot-object pair, we choose the target grasp that is closest to the sampled initial state. The matching cost combines palm-translation and palm-rotation differences. By pairing each sampled initial state with a nearby target grasp, the model is trained to move the hand toward a compatible grasp configuration from diverse starting poses. At inference time, no target grasp is available, so we sample the initial palm approach direction uniformly on the sphere while keeping the same open-hand joint initialization.

3) *Flow-matching objective*: We sample $t \sim \mathcal{U}(0, 1)$ and linearly interpolate between the initial and target states, yielding the target conditional velocity

$$x_t = (1 - t)x_0 + tx_1, \quad v^* = x_1 - x_0. \quad (4)$$

This standard linear conditional path is defined in the continuous grasp-state representation, using a continuous 6D

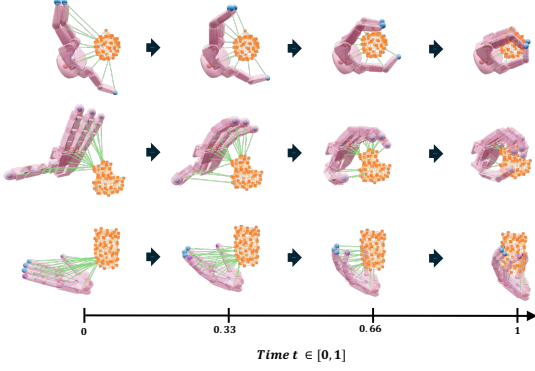


Fig. 4. State-dependent grasp generation. Starting from an open-hand state, GraspGraphNet repeatedly updates the palm pose and joint configuration. At each step, differentiable forward kinematics recomputes the articulated hand geometry, and the object-robot world edges are rebuilt using the current link positions.

orientation representation and joint-limit-normalized joint values. GraspGraphNet predicts $v_\theta(x_t, t)$, and we optimize the velocity-matching objective

$$\mathcal{L} = \text{MSE}(v_{\text{palm}}, v_{\text{palm}}^*) + \lambda_q \text{MSE}(v_q, v_q^*). \quad (5)$$

Here, v_{palm}^* and v_q^* are the palm and joint components of v^* , and λ_q balances their supervision.

F. Grasp Generation

At inference time, we sample an initial open-hand state $x^{(0)}$ and integrate the learned velocity field using K Euler steps:

$$\begin{aligned} x^{(k+1)} &= x^{(k)} + \Delta t v_\theta(x^{(k)}, t_k), \\ t_k &= \frac{k}{K}, \quad \Delta t = \frac{1}{K}. \end{aligned} \quad (6)$$

At every step, the updated grasp state changes the articulated hand geometry and therefore the dynamic object-robot world edges. Unless otherwise specified, we use $K = 3$, which the integration-step ablation shows to provide a favorable trade-off between grasp success and inference time.

The final state $x^{(K)}$ specifies a palm pose and normalized actuated-joint configuration. For execution, the canonical palm orientation is converted to the robot-native frame, the fixed palm-to-wrist offset defined by the robot model is applied, and the joint values are denormalized using their URDF limits. This produces an executable wrist pose and physical joint commands for the input robot hand.

IV. EXPERIMENTS

We evaluate GraspGraphNet in both simulation and real-world experiments. We measure grasp success, inference efficiency, and robustness to controlled hand-topology changes. We further ablate object-robot fusion, dynamic world-edge updates, integration steps, and the flow-matching objective.



Fig. 5. Evaluation object set used in simulation. The set contains 40 unseen objects, including 10 held-out CMapDataset objects and 30 objects from Google Scanned Objects.

A. Experimental Setup

a) Training Dataset: GraspGraphNet is trained on filtered CMapDataset [3], following prior cross-embodiment dexterous grasping works [11], [12]. The dataset contains object point clouds and grasp annotations for multiple dexterous robot hands. A GraspGraphNet model is trained using Barrett Hand, Allegro Hand, and Shadow Hand. For fair comparison, all learning-based baselines are trained under the same three-hand setting, using Barrett Hand, Allegro Hand, and Shadow Hand data from the same CMapDataset training split.

b) Evaluation Metrics: Generated grasps are evaluated in Isaac Gym using the same simulation-based protocol as DRO-Grasp [11] and TRO-Grasp, [12]. The evaluation object set consists of 40 unseen objects, including 10 objects held out from CMapDataset and 30 objects selected from Google Scanned Objects [18]. Fig. 5 shows the full set of evaluation objects. For each robot hand, 100 grasp trials are generated for each object, resulting in 4000 evaluation trials per hand.

A grasp is considered successful if the object displacement remains below 2 cm under six directional perturbations in Isaac Gym. The success rate is reported as the percentage of successful trials over all generated grasps. Inference efficiency is measured as the computational time required to generate one executable grasp command, excluding physics simulation. For a fair comparison, we re-evaluate all baseline methods using the same object set, simulation environment, evaluation protocol, and hardware.

c) Implementation Details: GraspGraphNet uses a hidden dimension of 128 and approximately 1.58 million parameters. The object graph contains four levels with 1024, 256, 64, and 32 points, while each robot link is represented by 64 sampled points. We use three robot-graph layers, two world-edge message-passing layers, and $k = 8$ nearest object points per robot link. The model is trained for 200 epochs using AdamW with an initial learning rate of 3×10^{-4} , cosine decay, and $\lambda_q = 1$. For initial-state sampling and target-grasp selection, we use a cone angle of 30° and a rotation-cost weight of 0.5, respectively. Unless otherwise specified, inference uses $K = 3$ Euler steps. All experiments are conducted on a single NVIDIA RTX 4090 GPU.

B. Overall Performance

Table I reports grasp success rate and inference time on Barrett Hand, Allegro Hand, and Shadow Hand. GraspGraphNet achieves the highest average success rate among all

TABLE I
COMPARISON OF SUCCESS RATE AND EFFICIENCY ACROSS METHODS.

Method	Success Rate (%) $\uparrow$				Efficiency (sec.) $\downarrow$			
	Barrett	Allegro	ShadowHand	Avg.	Barrett	Allegro	ShadowHand	Avg.
GenDexGrasp [3]	38.57(1543/4000)	19.72(789/4000)	28.87(1155/4000)	29.05	11.07	19.75	15.84	15.55
DRO-Grasp [11]	85.65(3426/4000)	81.07(3243/4000)	66.07(2643/4000)	77.60	0.583	0.421	0.703	0.569
TRO-Grasp [12]	82.12(3285/4000)	86.57(3463/4000)	74.30(2972/4000)	81.00	0.264	0.265	0.267	0.265
GraspGraphNet (Ours)	86.05(3442/4000)	87.75(3510/4000)	76.65(3066/4000)	83.48	0.024	0.033	0.063	0.040

compared methods, reaching 83.48%. Compared with recent cross-embodiment baselines, this improves the average success rate by 5.88 percentage points over DRO-Grasp and 2.48 percentage points over TRO-Grasp. GenDexGrasp performs lower in this evaluation, suggesting that recovering executable robot-specific grasps from hand-agnostic contact maps remains challenging across diverse hand embodiments. Across individual hands, GraspGraphNet remains competitive or superior on all three embodiments, achieving the best success rate on Barrett Hand, Allegro Hand, and Shadow Hand.

GraspGraphNet also provides a substantial improvement in inference efficiency. GenDexGrasp has the highest inference cost because each grasp requires contact-map-guided particle optimization. DRO-Grasp predicts an interaction representation rather than executable robot states, requiring additional reconstruction and optimization stages to obtain robot-specific grasp configurations. TRO-Grasp generates intermediate robot-object transformation graphs through diffusion and subsequently solves inverse kinematics to recover executable joint configurations. In contrast, GraspGraphNet directly integrates a learned velocity field over the executable palm and joint state, producing robot-specific grasp commands without a separate reconstruction stage. As a result, GraspGraphNet requires only 0.040 sec. per grasp on average, compared with 15.55 sec. for GenDexGrasp, 0.569 sec. for DRO-Grasp, and 0.265 sec. for TRO-Grasp. This corresponds to approximately $14\times$ faster inference than DRO-Grasp and $6.6\times$ faster inference than TRO-Grasp under the same evaluation setting. These results indicate that GraspGraphNet achieves both strong grasp success and high inference efficiency across multiple dexterous robot hand embodiments.

C. Generalization to Topology-Modified Hands

To evaluate generalization under hand-topology modifications, we create finger-ablated robot hands by removing one finger from the original hand morphology. For the Allegro Hand, three modified variants are created by removing each of the three fingers individually from the original four-finger hand. For the Shadow Hand, four modified variants are created by removing each of the four fingers individually from the original five-finger hand. The resulting modified hand topologies are illustrated in Fig. 6. These modified hand topologies are excluded from training for all methods. We evaluate all methods using the same trained models as in Sec. IV-B without additional retraining or adaptation. Removing a finger changes the URDF kinematic tree, the number of links, and the number of actuated joints, making this setting a direct test of robustness to topology modifications in the robot hand embodiment.

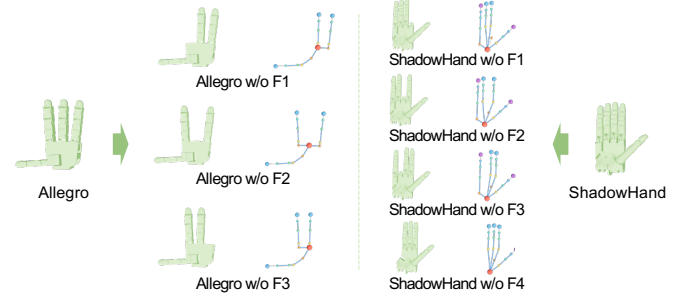


Fig. 6. Finger-ablated robot hands used for topology generalization. Each modified hand is represented by a new URDF-derived kinematic graph with different graph size and connectivity.



Fig. 7. Qualitative grasp results on topology-modified robot hands.

Table II reports success rates on the finger-ablated hands. GraspGraphNet achieves the highest overall success rate of 72.70%, outperforming DRO-Grasp by 15.92 percentage points and substantially outperforming GenDexGrasp and TRO-Grasp.

GenDexGrasp exhibits a substantial performance drop under finger removal. Because grasp generation is guided by predicted contact maps, some contact patterns learned on the original hand may become difficult to realize after changes in the available fingers and contact structure. TRO-Grasp also shows a noticeable decrease in performance. Its graph-based transformation representation is defined over object patches and hand links, making the learned interaction structure more sensitive to changes in the underlying hand topology. In contrast, DRO-Grasp remains relatively robust, suggesting that dense robot-object distance relationships provide a more transferable interaction representation under moderate topology variations.

GraspGraphNet achieves the best performance in five of the

TABLE II
COMPARISON OF SUCCESS RATES UNDER FINGER-REMOVAL SETTINGS ACROSS METHODS.

Method	Allegro			ShadowHand				Overall Avg.
	w/o F1	w/o F2	w/o F3	w/o F1	w/o F2	w/o F3	w/o F4	
GenDexGrasp [3]	20.75	24.02	17.00	6.95	8.30	7.55	6.37	12.99
DRO-Grasp [11]	63.00	64.90	58.15	43.22	47.40	62.75	58.05	56.78
TRO-Grasp [12]	21.47	18.80	9.10	5.92	9.60	30.25	11.95	15.29
GraspGraphNet (Ours)	87.40	84.07	57.45	73.72	75.12	75.80	55.35	72.70

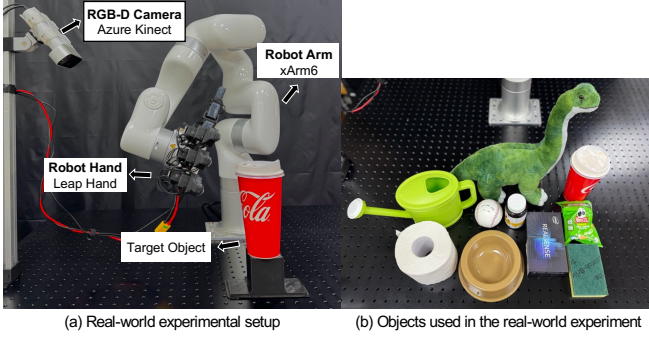


Fig. 8. Real-world experimental setup and test objects. (a) Experimental setup with the RGB-D camera, xArm6 robot arm, and Leap Hand. (b) Objects used in the real-world experiments.

seven finger-removal settings and remains competitive in the remaining two, where the gap to DRO-Grasp is less than 3 percentage points. Since robot embodiments are represented as URDF-derived kinematic graphs, changes in the number of links, joints, and graph connectivity can be directly incorporated into the model input without modifying the network architecture. This enables the learned grasp generation process to better accommodate topology variations across robot hands. Qualitative results in Fig. 7 further illustrate that GraspGraphNet remains effective despite changes in hand topology.

D. Real-World Experiments

We evaluate GraspGraphNet in real-world experiments, as shown in Fig. 8. The system consists of a UFactory xArm6 robot arm equipped with a Leap Hand [28] and an Azure Kinect RGB-D camera. For each trial, a partial point cloud of the target object is obtained from the RGB-D camera and provided as input to GraspGraphNet. The generated executable palm pose and joint commands are then directly executed on the robot. For real-world deployment, we fine-tune GraspGraphNet using the Leap Hand training data released by TRO-Grasp [12]. We evaluate GraspGraphNet on 10 objects with diverse shapes and sizes, shown in Fig. 8(b). For each object, we perform 10 grasp trials, resulting in 100 real-world grasp attempts in total.

A trial is considered successful if the robot grasps the object, lifts it from the table, and maintains the grasp without dropping it. GraspGraphNet achieves a success rate of 91%, demonstrating robust grasp generation from partial real-world point clouds and direct execution on a physical robot.

E. Ablation Study

TABLE III

ABLATION STUDY ON THE GENERATION OBJECTIVE. DIRECT REGRESSION USES THE SAME GRAPH ARCHITECTURE BUT PREDICTS THE FINAL GRASP STATE IN ONE FORWARD PASS.

Method	Success Rate (%)			
	Barrett	Allegro	ShadowHand	Avg.
Direct Regression	62.45	61.15	62.33	61.98
Flow Matching ($K = 1$)	82.75	88.02	66.05	78.94
Flow Matching ($K = 3$)	86.05	87.75	76.65	83.48

TABLE IV

ABLATION STUDY ON WORLD-EDGE MESSAGE PASSING AND CROSS-ATTENTION.

Method	Barrett	Allegro	ShadowHand	Avg.
w/o Cross-Attention	72.90	48.37	43.15	54.80
w/o World-Edge MPNN	52.30	50.05	51.47	51.27
Full Model	86.05	87.75	76.65	83.48

1) *Flow Matching vs. Direct Regression*: We compare conditional flow matching with a direct-regression variant under the same experimental setting, changing only the generation objective. The direct-regression model predicts the final displacement from the initial hand state in a single forward pass, rather than learning a time-conditioned velocity field.

As shown in Table III, direct regression achieves an average success rate of 61.98%, substantially lower than flow matching with one Euler step (78.94%) and three steps (83.48%). This suggests that dexterous grasp generation is difficult to model as a single-step regression problem, since multiple palm poses and joint configurations may correspond to valid grasps for the same robot-object pair. Direct regression can therefore produce averaged or imprecise grasp configurations. In contrast, flow matching models grasp generation as a sequence of state-conditioned updates, allowing the hand pose and joint configuration to be progressively refined toward stable grasps.

2) *Object-Robot Fusion Modules*: We ablate the two object-robot fusion modules in GraspGraphNet: cross-attention and world-edge message passing. The results are shown in Table IV. Removing cross-attention significantly reduces the average success rate from 83.48% to 54.80%. This indicates that global object context is important for conditioning robot node features on the overall shape of the target object. Removing the World-Edge MPNN further reduces the average success rate to 51.27%, demonstrating the importance of modeling local hand-object geometric interactions during grasp generation. The full model, which combines cross-attention and dynamic world-edge message passing, achieves the best performance across all three robot hands. These results suggest that the two fusion modules play complementary roles:

TABLE V
ABLATION STUDY ON DYNAMIC WORLD-EDGE MESSAGE PASSING.

Method	Success Rate (%)			
	Barrett	Allegro	ShadowHand	Avg.
Static world edges	72.52	42.55	35.62	50.23
Dynamic world edges	86.05	87.75	76.65	83.48

TABLE VI
EFFECT OF INTEGRATION STEPS K ON SUCCESS RATE AND INFERENCE TIME.

K	Success Rate (%)				Efficiency (sec.)
	Barrett	Allegro	ShadowHand	Avg.	
1	82.75	88.02	66.05	78.94	0.009
2	84.95	88.25	74.00	82.40	0.019
3	86.05	87.75	76.65	83.48	0.040
5	85.97	88.05	76.07	83.36	0.047
10	86.87	87.70	76.45	83.67	0.095
20	87.07	88.65	75.90	83.87	0.189

cross-attention provides global object context, while world-edge message passing captures local contact-relevant geometry in the shared world frame.

3) *Effect of dynamic world edges*: We evaluate the importance of dynamic world-edge construction using a static-edge variant, where object-link k NN assignments are computed once from the initial open-hand state x_0 and reused throughout flow integration. All other components remain unchanged, isolating the effect of updating robot-object neighbor connectivity. As shown in Table V, static world edges reduce the average success rate from 83.48% to 50.23%, with especially large drops for Allegro Hand and Shadow Hand. This indicates that fixed interaction neighborhoods cannot reliably track the changing local robot-object geometry during grasp formation. The results highlight the importance of updating robot-object interaction neighborhoods as the hand configuration evolves throughout grasp generation.

4) *Integration Steps K* : We evaluate the effect of the number of Euler integration steps used during inference. As shown in Table VI, increasing K from 1 to 3 improves the average success rate from 78.94% to 83.48%. This suggests that multiple flow updates help refine the palm pose and joint configuration as the hand approaches the object. However, further increasing K provides only marginal gains. Increasing K from 3 to 20 improves the average success rate by only 0.39 percentage points, while increasing inference time from 0.040 sec. to 0.189 sec. per grasp. We therefore use $K = 3$ in all main experiments as a practical trade-off between grasp quality and inference efficiency.

V. CONCLUSION

In this paper, we presented GraspGraphNet, a graph-based framework for cross-embodiment dexterous grasp generation. By representing robot hands as URDF-derived kinematic graphs and modeling robot-object interactions through dynamic world-edge message passing, GraspGraphNet learns a unified grasp generation policy across multiple dexterous hand embodiments with different kinematic structures and command spaces. To address the multimodality of dexterous

grasp generation, we formulate grasp synthesis as a conditional flow matching problem that iteratively refines palm poses and joint configurations in an executable grasp state space. Unlike prior approaches that rely on contact-based representations or grasp targets requiring additional optimization, retargeting, or inverse kinematics, GraspGraphNet directly generates executable palm poses and joint commands. Experimental results demonstrate strong performance across multiple dexterous hands, robustness to topology variations, and successful deployment on a real-world robotic system. Future work will investigate zero-shot generalization beyond topology variants to robot hands with substantially different morphologies, and extend the framework to broader dexterous manipulation tasks.

REFERENCES

- [1] X. Song, Y. Li, Y. Zhang, Y. Liu, and L. Jiang, "An overview of learning-based dexterous grasping: recent advances and future directions," *Artificial Intelligence Review*, vol. 58, no. 10, p. 300, 2025.
- [2] R. Wang, J. Zhang, J. Chen, Y. Xu, P. Li, T. Liu, and H. Wang, "Dexgraspnet: A large-scale robotic dexterous grasp dataset for general objects based on simulation," *arXiv preprint arXiv:2210.02697*, 2022.
- [3] P. Li, T. Liu, Y. Li, Y. Geng, Y. Zhu, Y. Yang, and S. Huang, "Gendexgrasp: Generalizable dexterous grasping," in *ICRA*. IEEE, 2023.
- [4] J. Zhang, H. Liu, D. Li, X. Yu, H. Geng, Y. Ding, J. Chen, and H. Wang, "Dexgraspnet 2.0: Learning generative dexterous grasping in large-scale synthetic cluttered scenes," in *CoRL*, 2024.
- [5] Z. Xu, C. Gao, Z. Liu, G. Yang, C. Tie, H. Zheng, H. Zhou, W. Peng, D. Wang, T. Hu *et al.*, "Manifoundation model for general-purpose robotic manipulation of contact synthesis with arbitrary objects and robots," in *IROS*. IEEE, 2024.
- [6] S. Brahmbhatt, A. Handa, J. Hays, and D. Fox, "Contactgrasp: Functional multi-finger grasp synthesis from contact," in *IROS*. IEEE, 2019.
- [7] A. Lakshminpathy, D. Bauer, C. Bauer, and N. S. Pollard, "Contact transfer: A direct, user-driven method for human to robot transfer of grasps and manipulations," in *ICRA*. IEEE, 2022.
- [8] H.-S. Fang, H. Yan, Z. Tang, H. Fang, C. Wang, and C. Lu, "Anydex-grasp: General dexterous grasping for different hands with human-level learning efficiency," *arXiv preprint arXiv:2502.16420*, 2025.
- [9] Z. Wu, R. A. Potamias, X. Zhang, Z. Zhang, J. Deng, and S. Luo, "Cedex: Cross-embodiment dexterous grasp generation at scale from human-like contact representations," *arXiv preprint arXiv:2509.24661*, 2025.
- [10] N. Khargonkar, L. F. Casas, B. Prabhakaran, and Y. Xiang, "Robotfinger-print: Unified gripper coordinate space for multi-gripper grasp synthesis and transfer," in *IROS*. IEEE, 2025.
- [11] Z. Wei, Z. Xu, J. Guo, Y. Hou, C. Gao, Z. Cai, J. Luo, and L. Shao, " $\mathcal{D}(\mathcal{R}, \mathcal{O})$ grasp: A unified representation of robot and object interaction for cross-embodiment dexterous grasping," in *ICRA*, 2025.
- [12] X. Fei, Z. Xu, H. Fang, T. Zhang, and L. Shao, "T(r,o) grasp: Efficient graph diffusion of robot-object spatial transformation for cross-embodiment dexterous grasping," 2025. [Online]. Available: <https://arxiv.org/abs/2510.12724>
- [13] D. Tola and P. Corke, "Understanding urdf: A dataset and analysis," *RA-L*, 2024.
- [14] P. W. Battaglia, J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, V. Zambaldi, M. Malinowski, A. Tacchetti, D. Raposo, A. Santoro, R. Faulkner *et al.*, "Relational inductive biases, deep learning, and graph networks," *arXiv preprint arXiv:1806.01261*, 2018.
- [15] T. Wang, R. Liao, J. Ba, and S. Fidler, "Nervenet: Learning structured policy with graph neural networks," in *ICLR*, 2018.
- [16] A. Patel and S. Song, "Get-zero: Graph embodiment transformer for zero-shot embodiment generalization," in *ICRA*. IEEE, 2025.
- [17] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," *arXiv preprint arXiv:2210.02747*, 2022.
- [18] L. Downs, A. Francis, N. Koenig, B. Kinman, R. Hickman, K. Reymann, T. B. McHugh, and V. Vanhoucke, "Google scanned objects: A high-quality dataset of 3d scanned household items," in *ICRA*. IEEE, 2022.
- [19] T. Liu, Z. Liu, Z. Jiao, Y. Zhu, and S.-C. Zhu, "Synthesizing diverse and physically stable grasps with arbitrary hand structures using differentiable force closure estimator," *RA-L*, 2021.

- [20] A. T. Miller and P. K. Allen, “Graspt! a versatile simulator for robotic grasping,” *IEEE Robotics & Automation Magazine*, vol. 11, no. 4, pp. 110–122, 2004.
- [21] Y. Xu, W. Wan, J. Zhang, H. Liu, Z. Shan, H. Shen, R. Wang, H. Geng, Y. Weng, J. Chen *et al.*, “Unidexgrasp: Universal robotic dexterous grasping via learning diverse proposal generation and goal-conditioned policy,” in *CVPR*, 2023.
- [22] W. Wan, H. Geng, Y. Liu, Z. Shan, Y. Yang, L. Yi, and H. Wang, “Unidexgrasp++: Improving dexterous grasping policy learning via geometry-aware curriculum and iterative generalist-specialist learning,” in *ICCV*, 2023.
- [23] J. Ye, K. Wang, C. Yuan, R. Yang, Y. Li, J. Zhu, Y. Qin, X. Zou, and X. Wang, “Dex1b: Learning with 1b demonstrations for dexterous manipulation,” *arXiv preprint arXiv:2506.17198*, 2025.
- [24] Q. Liu, Y. Cui, Q. Ye, Z. Sun, H. Li, G. Li, L. Shao, and J. Chen, “Dexrepnet: Learning dexterous robotic grasping network with geometric and spatial hand-object representations,” in *IROS*. IEEE, 2023.
- [25] L. Shao, F. Ferreira, M. Jorda, V. Nambiar, J. Luo, E. Solowjow, J. A. Ojea, O. Khatib, and J. Bohg, “Unigrasp: Learning a unified model to grasp with multifingered robotic hands,” *RA-L*, 2020.
- [26] M. Attarian, M. A. Asif, J. Liu, R. Hari, A. Garg, I. Gilitschenski, and J. Tompson, “Geometry matching for multi-embodiment grasping,” in *CoRL*. PMLR, 2023.
- [27] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “Pointnet++: Deep hierarchical feature learning on point sets in a metric space,” 2017.
- [28] K. Shaw, A. Agarwal, and D. Pathak, “Leap hand: Low-cost, efficient, and anthropomorphic hand for robot learning,” *RSS*, 2023.